

Lab 12 — Heaps and Priority Queues

DSA27 Lab Manual · Week 12 · Data Structures and Algorithms

Dr. Haitham A. El-Ghareeb

Fall 2026 · Week 12

Contents

Part 0 — Before you start	2
Part 1 — The array is the tree	4
Part 2 — <code>is_valid</code> and <code>peek</code>	5
Part 3 — <code>_sift_up</code> and <code>push</code>	6
Part 4 — <code>_sift_down</code> and <code>pop</code>	8
Part 5 — <code>heapify</code>	10
Part 6 — Break it: <code>MaxHeap</code> for free	11
Part 7 — <code>PriorityQueue</code> : a heap of triples	12
Part 8 — Measure it: <code>heapify</code> against <code>n</code> pushes	13
Part 9 — Check your heap against <code>heapq</code>	15
Part 10 — Exercises at a glance	16
Part 11 — Take-home practice (not graded)	17
Part 12 — Bridge to Lecture 13: hash tables	17
Summary	18
Answers to the checkpoints	18

How to use this lab. Read the matching section of Lecture 12 before each part. **Draw before you code:** every heap in this lab has at most ten values, and you should draw each one twice — as a tree and as the array under it. At each **Checkpoint**, write your prediction down first (answers at the end). Build `dsa/heap.py` one method at a time, in the order of the parts, and run the tests named in each part — the order is chosen so that each part's tests can pass before the next part is written. Parts 1–6 are the session; Parts 7–10 are for home.

Duration	One 2-hour lab session, plus about 3 hours at home
You will write	dsa/heap.py — MinHeap.is_valid, peek, _sift_up, push, _sift_down, pop, heapify; PriorityQueue.enqueue, dequeue, peek; and, for yourself, a small CountingMinHeap class
Graded by	tests/test_heap.py (28 tests)
Connects to	Lecture 12 — Heaps and priority queues; Lecture 04 (your DynamicArray); Lecture 10 (heap sort); Lecture 13 — Hash tables

What you will be able to do

1. move between a heap drawn as a tree and the same heap as an array, using $2i + 1$, $2i + 2$ and $(i - 1) // 2$;
 2. check the heap property on an array, edge by edge, in $O(n)$;
 3. write sift-up and sift-down, and trace both as a column of arrays;
 4. explain why sift-down must swap with the **better** child, and show an input on which “any child that beats it” goes wrong;
 5. write heapify bottom-up and say why the loop must run **backwards**;
 6. write every comparison through _beats, so that MaxHeap works with no code of its own — and predict which tests fail when you do not;
 7. build a priority queue on top of the heap, with a tie-breaking counter;
 8. count comparisons with a subclass, and show heapify’s $O(n)$ against the $O(n \log n)$ of n pushes.
-

Part 0 — Before you start

0.1 Environment and prerequisite

From the DSA27 folder, with the virtual environment active (the prompt starts with (.venv)):

```
git pull
pytest -m "not challenge" -q          # the environment check: must pass
pytest tests/test_dynamic_array.py -q # your Week 4 DynamicArray: must pass
pytest tests/test_heap.py -q          # 28 failures: nothing written yet
```

The heap stores its values in **your** DynamicArray from Week 4, and uses four of its operations: append (push), pop() with no argument (removing the last item in pop), self._items[i] for reading and self._items[i] = x for writing. If test_dynamic_array.py fails, fix that first: a broken __setitem__ makes every swap in

this lab look like a heap bug.

The 28 failures are all `NotImplementedError` — that is the starting line.

0.2 Read the skeleton: what is given, what is yours

Open `dsa/heap.py`. Read all of it before you write anything.

Class	Given to you	Yours
MinHeap	<code>__init__(values=())</code> — copies the values into <code>self._items</code> , a <code>DynamicArray</code> , and calls <code>heapify</code> if there are any; <code>_beats</code> ; <code>is_empty</code> , <code>__len__</code> , <code>__repr__</code>	<code>_sift_up</code> , <code>_sift_down</code> , <code>push</code> , <code>pop</code> , <code>peek</code> , <code>heapify</code> , <code>is_valid</code>
MaxHeap	everything: it overrides <code>_beats</code> only	nothing — if your <code>MinHeap</code> is written right
PriorityQueue	<code>__init__</code> — <code>self._heap</code> , a <code>MinHeap</code> , and <code>self._counter = 0</code> ; <code>is_empty</code> , <code>__len__</code>	<code>enqueue</code> , <code>dequeue</code> , <code>peek</code>

Read `_beats` twice:

```
def _beats(self, child, parent):
    return child < parent
```

It answers one question: **must child move above parent?** In a min-heap, yes when it is smaller; in `MaxHeap`, whose `_beats` says `child > parent`, yes when it is larger. Every comparison between two values in your code must be a call to `self._beats(...)`, never a `<` or `>` of your own. That is how one class gives you both heaps (Lecture 12, “Order: the heap property”).

Notice also what `__init__` does: `MinHeap([5, 3, 8])` calls **your** `heapify`. Until Part 5, build heaps in the REPL with `MinHeap()` and `push`, not from a list.

0.3 The storage rule, this week

A `dsa/` structure stores its data only in the course `Array`, in node objects, or in another `dsa/` structure — never in a Python `list`, `dict` or `set` (Lecture 02). Here:

- `MinHeap` keeps everything in `self._items`, your `DynamicArray`. There is no second structure: the tree **is** the array.
- `PriorityQueue` keeps everything in `self._heap`, a `MinHeap`. The triples it pushes, `(priority, counter, item)`, are **values** — one small record each — not storage.
- `list(heap._items)` for drawing or printing is fine: it is output, not storage.

0.4 The tests, and one warning about -k

Eleven test functions run twice, once with [MinHeap] and once with [MaxHeap] in their names; one more compares the two classes; five test the priority queue. That is 28.

-k matches the file name too, and the file is test_heap.py. So -k heap selects **all 28** tests. The filters this lab uses avoid the word:

Filter	Selects
-k "push_then_peek or after_every_push"	4 — Part 3
-k "empty_heap or single_element or interleaved"	6 — Part 4
-k "not priority"	23 — every MinHeap/MaxHeap test, Part 5
-k MaxHeap	the 11 [MaxHeap] cases — Part 6
-k priority	the 5 priority-queue tests — Part 7

Part 1 — The array is the tree

Lecture 12, “The array is the tree”. Nothing to write yet: this part is paper and the REPL.

1.1 The idea

Number a complete binary tree level by level, left to right, from 0. Node i lives in slot i . Then:

```
left(i)   = 2*i + 1
right(i)  = 2*i + 2
parent(i) = (i - 1) // 2      for i > 0
```

A child index that is $\geq n$ does not exist. The leaves are the back half of the array, from $n // 2$ on; the last node with a child is $n // 2 - 1$.

1.2 Draw it

Draw [2, 4, 3, 9, 7, 8, 5, 12, 10] as a tree on paper, with each node’s index beside it. Then check your drawing (paste into notebooks/12-heaps.ipynb):

```
from viz.draw import draw_array_as_tree
draw_array_as_tree([2, 4, 3, 9, 7, 8, 5, 12, 10], highlight=1)
```

draw_array_as_tree knows nothing about heaps. It draws any list as a tree — which makes it the debugging tool of the week: whenever a test fails, draw `list(h._items)` and look for a parent that a child beats.

1.3 Checkpoint

Checkpoint 1. A heap holds 12 values, at indices 0 to 11.

- (a) What are the children of index 4? Of index 5?
 - (b) What is the parent of index 11? Of index 1?
 - (c) Which index is the **last parent** — the last node with at least one child? Which indices are leaves?
 - (d) What is the height of the tree (the root alone has height 0), and on which level is index 7?
 - (e) Index 5 has a left child. Does it have a right child?
-

Part 2 — is_valid and peek

Lecture 12, “Order: the heap property”.

2.1 The idea

The heap property is a statement about **edges**: for every parent–child pair, the child does not beat the parent. Every node except the root has exactly one parent, so there are exactly $n - 1$ edges — and walking over the **children**, from index 1 to $n - 1$, visits each edge once.

peek is the reason a heap exists: the best value is at index 0.

2.2 Draw it

For each array, draw the tree, and put a cross on every edge where a child beats its parent (as a min-heap):

[1, 3, 2, 7, 4]	[1, 3, 2, 2, 4]	[5, 5, 5]	[3, 4, 5, 1]
-----------------	-----------------	-----------	--------------

2.3 Checkpoint

Checkpoint 2. (a) Which of the four arrays above are valid min-heaps? (b) How many calls to `_beats` does a correct `is_valid` make on a **valid** heap of n values? On an invalid one, can it make fewer? (c) A student's `is_valid` checks only the root against its two children. Which of the four arrays does it get wrong?

2.4 Write it

is_valid:

- For every child index from 1 to `len(self._items) - 1`: compute its parent; if `self._beats(child's value, parent's value)`, the property is broken — return `False`.
- After the loop, return `True`.

peek: raise `IndexError` if the heap is empty; otherwise return the value at index 0.

2.5 Test it — by hand

No test can run `is_valid` yet: they all build their heaps with `push` or `heapify`. So test it at the REPL, by filling `_items` directly:

```
>>> from dsa.heap import MinHeap, MaxHeap
>>> h = MinHeap()
>>> for v in [1, 3, 2, 2, 4]:
...     h._items.append(v)
...
>>> h.is_valid(), h.peak()
(False, 1)
```

Try all four arrays of 2.2, and one of them in a `MaxHeap` as well. Then `MinHeap().peek()` must raise `IndexError`.

2.6 When it fails

Bug	What you see	Fix
loop from 0: <code>range(len(self._items))</code>	index 0's "parent" is $(0 - 1) // 2 = -1$, which your <code>DynamicArray</code> reads as the last item: the root beats it, so <code>is_valid</code> rejects almost every valid heap	start at 1: the root has no parent
<code>items[child] < items[parent]</code> instead of <code>_beats</code> only the root's children checked	right for <code>MinHeap</code> ; for <code>MaxHeap</code> , it rejects every valid max-heap — once the rest is written, 5 [MaxHeap] tests fail nothing, for now: all 28 tests pass with this bug once the rest works	<code>self._beats(items[child], items[parent])</code> check every edge — see below
<code>peek</code> returns <code>None</code> when empty	DID NOT RAISE <code>IndexError</code> in the empty-heap test	raise <code>IndexError</code>

The third row matters most, because no test catches it: every heap the tests check is either valid, or broken at the root (`test_is_valid_rejects_a_broken_array` swaps index 0 with index 2). A validator that only looks at the top is a validator you cannot trust in Parts 3–5, where `is_valid` is how the tests catch **your** sifting bugs.

Part 3 — `_sift_up` and `push`

Lecture 12, "Sift Up: push".

3.1 The idea

A new value can only go in one place without breaking the shape: the end of the array. After that, the only edge that can be wrong is the one between the new value and its parent. Swap while it is wrong; each swap moves the problem one level up; stop at the root at the latest.

3.2 Draw it

Push 4, 9, 2, 7, 1, 5 into an empty min-heap. After **each** push, draw the tree, write the array under it, and mark each swap with an arrow.

3.3 Checkpoint

Checkpoint 3. (a) Give the array after each of the six pushes, and the number of swaps each push makes. (b) Push 1 made the most swaps. Could any push into a heap of 5 values make more? What is the most a push can make into a heap of n values? (c) Why does the sift-up stop at index 0 even if you forget to test for it — or does it?

3.4 Write it

_sift_up(index):

1. While index is not the root:
 - compute the parent's index;
 - if the item at index does **not** beat its parent, stop;
 - otherwise swap the two items, and continue from the parent's index.

push(value): append the value to `self._items`, then sift up from the last index.

Hints:

- A swap in Python is one line: `a[i], a[j] = a[j], a[i]`. It works on your `DynamicArray` because it has both `__getitem__` and `__setitem__`.
- Pass the arguments to `_beats` in the right order: the **child** first. For `MinHeap` the order decides the answer: `_beats(3, 5)` is `True`, `_beats(5, 3)` is `False`.

3.5 Test it

```
pytest tests/test_heap.py -v -k "push_then_peek or after_every_push"
```

Four tests: `test_push_then_peek_gives_the_best` pushes 5, 3, 8, 1, 9 and peeks; `test_invariant_holds_after_every_push` pushes 40 random values and calls `is_valid()` after each — for both classes.

3.6 When it fails

Bug	What you see	Fix
< in place of <code>_beats</code>	only the [MaxHeap] tests fail — <code>assert 1 == 9</code> in the [MaxHeap] peek test	always <code>self._beats(child, parent)</code>
<code>parent = index // 2</code> (the 1-based formula)	only the [MaxHeap] after-every-push test fails, with <code>assert False</code>	<code>(index - 1) // 2</code> — see below
<code>while index >= 0</code>	<code>(0 - 1) // 2</code> is <code>-1</code> , the last item: the root swaps with a leaf. <code>assert 3 == 1</code> in the [MinHeap] peek test	<code>while index > 0</code>
no swap, only a comparison	<code>assert 5 == 1</code> : nothing ever moves	swap, then move index to the parent

The second bug is the dangerous one. For odd indices `index // 2` happens to be right, and for even ones it names the node just **after** the true parent — so most pushes still work, and three of the four tests pass. On paper: `[1, 5, 2, 6]` plus a push of 3 at index 4 compares 3 with index 2 (value 2), stops, and leaves 3 under 5. Whenever you write an index formula, try it on indices 1, 2, 3 and 4 by hand.

Part 4 — `_sift_down` and `pop`

Lecture 12, “Sift Down: `pop`” and “The trap: swap with the **better** child”.

4.1 The idea

The root is the answer. To remove it without leaving a hole, take the **last** item — the only one whose slot can disappear — and put it at the root. Now the root may be beaten by a child. Swap it with its **better** child, which beats both it and its sibling, and continue below. Stop when no child beats it or it has no children.

4.2 Draw it

Start from the min-heap `[1, 4, 2, 7, 5, 3, 6]`. Draw it. Pop three times, drawing the tree after each pop: first the last item moved to the root, then each swap.

4.3 Checkpoint

Checkpoint 4. (a) Give each value returned and the array after each of the three pops. (b) A student’s sift-down swaps with the **first** child that beats the item — left if the left one does, otherwise right. Run the first pop that way: which array results, and what does the **second** pop return? (c) How many comparisons can one sift-down level make, and why not one?

4.4 Write it

`_sift_down(index):`

1. Loop:
 - `best = index`; compute left and right;
 - if left exists (it is `< len(self._items)`) and its item beats the item at best, set `best = left`;
 - if right exists and its item beats the item at best, set `best = right`;
 - if `best == index`, stop; otherwise swap the items at `index` and `best`, and continue from `best`.

`pop():`

1. Empty: raise `IndexError`.
2. Remove the last item with `self._items.pop()`.
3. If the heap is now empty, that item **was** the root: return it.
4. Otherwise save the root, write the last item into index 0, sift down from 0, and return the saved root.

Hints:

- Compare the right child with `items[best]`, not with `items[index]`. That one detail is what makes it the **better** child.
- `self._items.pop()` with no argument removes the **last** item, in $O(1)$. With argument 0 it would shift everything — and give the wrong answer too.

4.5 Test it

```
pytest tests/test_heap.py -v -k "empty_heap or single_element or interleaved"
```

Six tests: the empty heap (`pop` and `peek` raise `IndexError`), a single element pushed and popped, and `test_interleaved_push_and_pop`, which pushes 60 random values, pops every third time, checks each popped value against the true best, and calls `is_valid()` after every step — for both classes. It is the test that makes sift-up and sift-down work **together**.

4.6 When it fails

Bug	What you see	Fix
no one-item case (step 3)	<code>IndexError: 0</code> — raised by your DynamicArray , not by the heap — in the single-element test, and later in 7 more tests	after the removal, if the heap is empty, return the item
<code>right < size</code> check copied as <code>left < size</code>	<code>IndexError: 2, IndexError: 6, ... from DynamicArray.__getitem__</code> : the right child is read when it does not exist	each child needs its own bound

Bug	What you see	Fix
first child that beats, not the better one	these tests may pass; in Part 5, 6 tests fail, among them the after-every-pop test with <code>assert False</code>	compare right with <code>items[best]</code>
one swap, no loop	the heap is repaired one level deep only; 4 tests fail in Part 5	loop until <code>best == index</code>
return <code>self._items[0]</code>	the interleaved test fails with <code>assert 21 == 14</code> : you returned the new root	save the root before overwriting it
after the sift-down		
<code>self._items.pop(0)</code>	it returns the right value once, then the heap is broken: <code>assert False</code> from <code>is_valid</code>	the last item moves to the root
as the whole of pop		

Note what is **missing** from the table: a pop that forgets the empty check still passes `test_empty_heap`, because your `DynamicArray.pop` raises `IndexError` on its own. Write the check anyway: the message should say “empty heap”, not something about an array the caller has never heard of.

Part 5 — heapify

Lecture 12, “Build-Heap in $O(n)$ ”.

5.1 The idea

`MinHeap(values)` has the right shape already — every array does — and possibly no order at all. A leaf is a heap of one. Sift down every node that **has** a child, from the last one ($n // 2 - 1$) **back** to the root. When node i is sifted down, both of its subtrees are already heaps, which is exactly the situation sift-down repairs.

5.2 Draw it

Heapify `[8, 7, 6, 5, 4, 3, 2, 1]` on paper. Draw the tree once; then, for each index from the last parent back to 0, sift it down and redraw.

5.3 Checkpoint

Checkpoint 5. (a) Which index do you sift down first, and which last? (b) Give the array after each sift-down, and the total number of swaps. (c) Why must the loop run **backwards**? Give a three-level example of what goes wrong if you sift down index 0 first. (d) The loop could start at $n - 1$ instead of $n // 2 - 1$ and still be correct. Why? What does it waste?

5.4 Write it

One loop: for each index from `len(self._items) // 2 - 1` **down to and including** 0, call `self._sift_down(index)`.

A range with a negative step stops **before** its stop value, so the stop must be `-1` to include 0. That is the entire exercise, and the lecture's heap sort (Week 10) has the same line.

5.5 Test it

```
pytest tests/test_heap.py -v -k "not priority"
```

All 23 MinHeap and MaxHeap tests — every test that builds a heap from a list now runs, including `test_heapify_from_arbitrary_order` on `[9, 4, 7, 1, 8, 2, 6, 3, 5]`, the lecture's `heapify` trace.

5.6 When it fails

Bug	What you see	Fix
<code>range(n // 2 - 1, 0, -1)</code> — the root is skipped	5 tests fail; the max-differs-from-min test says <code>assert 5 == 1: the root was never sifted</code>	stop at <code>-1</code>
forward: <code>range(n // 2)</code>	5 tests fail, for example <code>assert [3, 1, 2, 5, 7, 8, ...] == [1, 2, 3, 5, 7, 8, ...]</code> — popping gives a wrong order	backwards: subtrees first
starting one too early, at <code>n // 2 - 2</code>	4 tests fail — the last parent is never sifted	start at <code>n // 2 - 1</code>
<code>n</code> pushes in disguise: <code>for i in range(1, n):</code> <code>self._sift_up(i)</code>	nothing: all 28 pass. It is correct, and $O(n \log n)$	Part 8 measures it — the docstring asks for $O(n)$

The last row is the one to think about. A forward `_sift_up` over every index is a correct heap construction — it is exactly `n` pushes, done in place — and no test can tell it from `heapify` by the answers alone. Only the **cost** differs. That is what Part 8 is for.

Part 6 — Break it: MaxHeap for free

Lecture 12, "Order: the heap property". Your MinHeap passes its tests; so, with no code of its own, does MaxHeap — **if** every comparison went through `_beats`. Check it, and then break it on purpose.

```
pytest tests/test_heap.py -v -k MaxHeap # 11 passed
```

Now, in `_sift_down` only, replace the two `self._beats(a, b)` calls by `a < b`. Leave everything else alone.

Checkpoint 6. Before you run anything:

- (a) Which class still passes every test, and why?
- (b) For `MaxHeap`, what does the broken sift-down do instead of what it should?
- (c) Of the 28 tests, how many fail? Name the kinds of test that fail, and say why `test_invariant_holds_after_every_push[MaxHeap]` still passes.

Now run the whole file and compare with your prediction:

```
pytest tests/test_heap.py -v
```

Then **put `_beats` back** and run it again: 28 passed. `git diff dsa/heap.py` shows what you have changed since your last commit, if you are not sure.

What to take from this: one class, two heaps, and a single point of truth for “better”. A comparison written anywhere else is a comparison `MaxHeap` does not know about.

Part 7 — PriorityQueue: a heap of triples

Lecture 12, “The Priority Queue on Top”.

7.1 The idea

Push `(priority, counter, item)` onto `self._heap`, then add 1 to `self._counter`. Python compares tuples left to right, so the priority decides; on a tie, the counter decides — and since every counter is different, the item itself is **never** compared. `dequeue` and `peek` return only the item: index 2 of the triple.

7.2 Checkpoint

Checkpoint 7. `enqueue("b", 2), enqueue("a", 2), enqueue("c", 1)`.

- (a) Give the heap array, as triples, after each enqueue.
- (b) In what order do three dequeues return the items?
- (c) If enqueue pushed `(priority, item)` instead, in what order would they come out? And what would happen with `enqueue({"task": "a"}, 1)` followed by `enqueue({"task": "b"}, 1)`?

7.3 Write it

- **`enqueue(item, priority)`**: push the triple onto the heap, then add 1 to the counter.

- **dequeue()**: empty $\rightarrow$ `IndexError`; otherwise pop a triple from `self._heap` and return its item.
- **peek()**: the same with peek.

Build on the heap — call its methods. Do not reach into `self._heap._items`, and do not sift anything here: that work is done.

7.4 Test it

```
pytest tests/test_heap.py -v -k priority
```

Five tests: serving the lowest number first; `test_priority_queue_is_not_a_fifo`, which enqueues 0..4 with priorities 9, 7, 5, 3, 1 and expects 4, 3, 2, 1, 0; peek and length; the empty queue; and two dictionaries with equal priority.

7.5 When it fails

Bug	What you see	Fix
(priority, item) pairs, and [2] in dequeue	<code>IndexError: tuple index out of range</code> in 3 tests, and <code>TypeError: '<' not supported between instances of 'dict' and 'dict' in the fourth</code>	push the triple
a counter in the triple, but never incremented	only the unorderedable-items test fails, with the same <code>TypeError</code> : every counter is 0, so a tie still reaches the items	<code>self._counter += 1</code> in enqueue
returning the whole triple	<code>assert (1, 1, 'fix the build') == 'fix the build'</code>	return index 2
peek returns None when empty	<code>DID NOT RAISE IndexError</code> in the empty-queue test	raise, as dequeue does

Part 8 — Measure it: heapify against n pushes

Lecture 12, “Measured: heapify against n pushes”. A timing depends on your machine; a **count** of comparisons does not. You will count them with a subclass that looks exactly like a `MinHeap` to your code.

8.1 Write CountingMinHeap

This is not graded, and it does not go in `dsa/`: write it in the scratch space of `notebooks/12-heaps.ipynb`.

```
import random
from dsa.heap import MinHeap
```

```

class CountingMinHeap(MinHeap):
    """A MinHeap that counts every comparison it makes."""

    def __init__(self, values=()):
        self.comparisons = 0      # first: __init__ runs heapify
        super().__init__(values)

    def _beats(self, child, parent):
        self.comparisons += 1
        return super()._beats(child, parent)

```

Because **all** your comparisons go through `_beats` (Part 6), overriding that one method sees every one of them. This is the payoff of the design, a second time.

Checkpoint 8. Predict, for $n = 1,024$, the comparisons **per element** for (a) heapify on descending input, (b) n pushes of descending input into a min-heap, (c) n pushes of random input. Then for $n = 16,384$. Which of the three grows with n ?

8.2 Count

```

def by_heapify(values):
    return CountingMinHeap(values).comparisons

def by_pushes(values):
    h = CountingMinHeap()
    for v in values:
        h.push(v)
    return h.comparisons

for n in [1024, 16384]:
    descending = list(range(n, 0, -1))
    shuffled = random.Random(n).sample(range(n), n)
    print(f'n = {n}')
    for name, values in [('descending', descending), ('random', shuffled)]:
        a = by_heapify(values) / n
        b = by_pushes(values) / n
        print(f' {name:10} heapify {a:5.2f}   pushes {b:5.2f}'
              ' comparisons per element')

```

With the reference solution:

n = 1024					
descending	heapify	1.98	pushes	8.01	comparisons per element
random	heapify	1.86	pushes	2.25	comparisons per element
n = 16384					
descending	heapify	2.00	pushes	12.00	comparisons per element
random	heapify	1.88	pushes	2.27	comparisons per element

Your counts should match to the last digit if your sifts compare exactly as Part 4.4 says. If your heapify shows **12.00** on descending input at $n = 16,384$, it is the n -pushes version from the last row of Part 5.6.

8.3 Time it

```
from viz.complexity import measure

def build_by_pushes(values):
    h = MinHeap()
    for v in values:
        h.push(v)

sizes = [2_000, 4_000, 8_000, 16_000, 32_000]
descending = lambda n: list(range(n, 0, -1))
_, t_heapify = measure(MinHeap, sizes, descending)
_, t_pushes = measure(build_by_pushes, sizes, descending)
for n, a, b in zip(sizes, t_heapify, t_pushes):
    print(f'{n:6}  heapify {a * 1e3:7.1f} ms  pushes {b * 1e3:7.1f} ms'
          f'    ratio {b / a:4.1f}')
```

`measure(MinHeap, ...)` times the constructor, which is your heapify plus the copy into the `DynamicArray`. On the instructor's machine the ratio grew from about 4.5 at $n = 2,000$ to about 7 at $n = 32,000$: the pushes grow like $n \log n$, heapify like n , so the gap keeps widening. Your milliseconds will differ; the widening will not.

Part 9 — Check your heap against heapq

Only now, with your own heap passing its tests, meet the library version (Lecture 12, “Where priority queues are used”). Python's `heapq` works on a plain list with exactly your index arithmetic. Use it as an **oracle** — a second opinion — on many random sequences of operations:

```
import heapq, random
from dsa.heap import MinHeap

rng = random.Random(1)
for trial in range(2_000):
```

```

mine, oracle = MinHeap(), []
for step in range(rng.randint(0, 40)):
    if oracle and rng.random() < 0.4:
        assert mine.pop() == heapq.heappop(oracle), (trial, step)
    else:
        v = rng.randint(0, 20)
        mine.push(v)
        heapq.heappush(oracle, v)
    assert mine.is_valid() and len(mine) == len(oracle)
values = [rng.randint(0, 20) for _ in range(rng.randint(0, 30))]
built = MinHeap(values)
assert sorted(built._items) == sorted(values) and built.is_valid()
print('2,000 random runs: all agree')

```

Small values (0 to 20) on purpose, so that there are many duplicates, which the graded tests touch only once. The two heaps need not hold the same **array** — there can be more than one valid heap of the same values — but every pop must return the same value. If an assertion fails, the message gives the trial and the step: rerun that trial alone and draw it.

This is also how you use `heapq` from now on: in your own programs, call the library; in `dsa/`, the point is that you can write it.

Part 10 — Exercises at a glance

Method	Target cost	The trap	-k filter
<code>is_valid</code>	$O(n)$	every edge, not just the root's; start at 1	by hand, Part 2
<code>peek</code>	$O(1)$	<code>IndexError</code> when empty	<code>empty_heap</code>
<code>_sift_up</code> , <code>push</code>	$O(\log n)$	<code>(i - 1) // 2</code> ; stop at the root; <code>_beats(child, parent)</code>	Part 3 (4)
<code>_sift_down</code> , <code>pop</code>	$O(\log n)$	the better child; each child's own bound; the one-item heap	Part 4 (6)
<code>heapify</code>	$O(n)$	backwards from <code>n // 2 - 1</code> to 0 inclusive; not <code>n</code> pushes	"not priority" (23)
<code>MaxHeap</code>	—	every comparison through <code>_beats</code>	<code>MaxHeap</code> (11)
<code>PriorityQueue</code>	$O(\log n)$	the triple; increment the counter; return the item	<code>priority</code> (5)

All 28 at once:

```
pytest tests/test_heap.py -v
```

Before you show the TA: `git diff --stat tests/` must print nothing. And check by eye that no `<` or `>` between two values appears anywhere in `MinHeap` except inside `_beats`, and that `is_valid` looks at every child — the tests cannot see either, but the TA will look.

Part 11 — Take-home practice (not graded)

The question bank for this week is `docs/question-bank/week12-questions.md`, with answers in `week12-answers.md`. Do the questions before opening the answers.

1. **Part G — write the code**, in `practice/week12.py`: `is_min_heap`, `top_k`, `merge_sorted`, `join_ropes` and `running_medians` (W12-C1 to W12-C5). Use your own `MinHeap` and `MaxHeap`; two of the tests count your comparisons, so sorting everything fails:

```
pytest tests/test_practice_week12.py -v
```

Hint for `running_medians`: a `MaxHeap` for the smaller half and a `MinHeap` for the larger half — Part 6 is why you have both for free.

2. **W12-T1** — Homework 12, item 2: push 6, 2, 8, 1, 5, 3; then pop twice.
3. **W12-T2** — heapify [3, 9, 2, 1, 4, 5, 8, 7, 6, 0] and count the swaps.
4. **W12-B2** — the first-child sift-down of Checkpoint 4, as a find-the-bug.
5. **W12-K3** — the forward heapify: find an input on which it fails without running it.

The worked solutions are in `solutions/dsa/heap.py` and `solutions/practice/week12.py` — for after you have tried. `pytest --solutions tests/test_practice_week12.py` runs the tests on them.

Part 12 — Bridge to Lecture 13: hash tables

A heap answers one question fast — *what is the best item?* — and every other question slowly. Try this at the REPL, with your finished heap:

```
from dsa.heap import MinHeap
h = MinHeap(range(1, 1001))
print(1000 in list(h._items))    # only by looking at all of it
```

There is no shortcut: 1000 could be in any leaf, and there are 500 of them. So two operations that real priority queues need are $O(n)$ here:

- **“Is this task already queued?”** — a search through the array.

- **“Make this task more urgent.”** — find it ($O(n)$), change its priority, then sift it up ($O(\log n)$). The sift is cheap; the **finding** is not. Dijkstra’s algorithm, which Lecture 12 named as beyond this course, does this millions of times.

The standard fix keeps, next to the heap, a second structure that maps each item to its current **index** in the array, updated on every swap. Finding a task then costs whatever that map costs to look up. With a sorted array and binary search it would be $O(\log n)$. Lecture 13 builds a structure where it is **$O(1)$** on average — the hash table — by computing where a key lives instead of searching for it. Bring one question: what does a hash table have to give up to be that fast?

Summary

Idea	The one line to keep
Shape	A complete binary tree: an array with no gaps, height $\lfloor \log_2 n \rfloor$.
Index arithmetic	Children $2i + 1, 2i + 2$; parent $(i - 1) // 2$; leaves from $n // 2$.
Heap property	Every child against its parent: $n - 1$ checks, starting at index 1.
push	Append, then sift up while the item beats its parent.
pop	Save the root, move the last item up, sift down; the one-item heap is special.
The trap	Sift down past the better child — compare the right with <code>items[best]</code> .
heapify	Sift down from $n // 2 - 1$ back to 0 inclusive: $O(n)$. n pushes are $O(n \log n)$.
<code>_beats</code>	One method decides “better”; <code>MaxHeap</code> overrides only it.
PriorityQueue	Push (<code>priority, counter, item</code>); the counter breaks ties, first come first served.
Measuring	Count comparisons by overriding <code>_beats</code> in a subclass.

Answers to the checkpoints

Checkpoint 1. (a) Index 4: children 9 and 10. Index 5: children 11 and 12 — but 12 does not exist ($n = 12$), so only 11. (b) Parent of 11: $(11 - 1) // 2 = 5$. Parent of 1: $(1 - 1) // 2 = 0$. (c) The last parent is $12 // 2 - 1 = 5$; the leaves are indices **6 to 11** — half the heap. (d) Height

$\lfloor \log_2 12 \rfloor = 3$. Index 7 is on level 3: level k starts at $2^k - 1$, and level 3 starts at index 7. (e) No: its right child would be 12, past the end. A heap has at most one node with a single child, and it is always the last parent when n is even.

Checkpoint 2. (a) $[1, 3, 2, 7, 4]$ — valid. $[1, 3, 2, 2, 4]$ — **not**: 2 at index 3 is below its parent 3. $[5, 5, 5]$ — valid: equal values never beat each other. $[3, 4, 5, 1]$ — **not**: 1 at index 3 is below 4. (b) Exactly $n - 1$: one per child, one per edge. On an invalid heap it can stop at the first violation, so it can make fewer. (c) It gets $[1, 3, 2, 2, 4]$ and $[3, 4, 5, 1]$ wrong — both break the property **below** the root's children, and it answers True for both.

Checkpoint 3. (a)

Push	Array	Swaps
4	[4]	0
9	[4, 9]	0
2	[2, 9, 4]	1
7	[2, 7, 4, 9]	1 — 7 swaps with 9, then stops under 2
1	[1, 2, 4, 9, 7]	2 — index 4 $\rightarrow$ 1 $\rightarrow$ 0
5	[1, 2, 4, 9, 7, 5]	0 — its parent is 4

(b) No: a heap of 5 has height 2, so a push into it (becoming the sixth item, still at depth 2) makes at most 2 swaps. Into a heap of n , at most $\lfloor \log_2(n + 1) \rfloor$ — the depth of the new slot.

(c) Only because of the loop condition. The parent formula at index 0 gives $(0 - 1) // 2 = -1$, and your `DynamicArray` reads index -1 as the last item — so a loop that runs at index 0 compares the root with a leaf and may swap them. `while index > 0` is not decoration.

Checkpoint 4. (a) Pop 1: 6 moves to the root, swaps with 2, then with 3 $\rightarrow$ $[2, 4, 3, 7, 5, 6]$. Pop 2: 6 to the root, swaps with 3 $\rightarrow$ $[3, 4, 6, 7, 5]$. Pop 3: 5 to the root, swaps with 4 $\rightarrow$ $[4, 5, 6, 7]$. Returned: **1, 2, 3**. (b) The first pop still returns 1, but 6 swaps with 4 (the left child, which beats 6) instead of 2, and then with 5: $[4, 5, 2, 7, 6, 3]$, with 4 above 2. The second pop returns **4** instead of 2. (c) Two: left child against the current best, then right child against the current best. One comparison could not tell which of the two children is better **and** whether it beats the item.

Checkpoint 5. (a) $n = 8$: first $8 // 2 - 1 = 3$, last **0**.

<code>sift_down(i)</code>	Array afterwards	Swaps
3 (value 5)	[8, 7, 6, 1, 4, 3, 2, 5]	1
2 (value 6)	[8, 7, 2, 1, 4, 3, 6, 5]	1
1 (value 7)	[8, 1, 2, 5, 4, 3, 6, 7]	2
0 (value 8)	[1, 4, 2, 5, 8, 3, 6, 7]	2

(b) **6 swaps** in all (and 10 comparisons).

- (c) Sift-down assumes that both subtrees below the node are heaps; it only repairs the node itself. Going backwards makes that true every time. Forwards, on $[5, 4, 3, 2, 1]$: sifting 0 first swaps 5 with 3 $\rightarrow [3, 4, 5, 2, 1]$; then index 1 swaps 4 with 1 $\rightarrow [3, 1, 5, 2, 4]$. Now 1 sits **below** 3, and nothing will ever look at the root again.
- (d) Every index from $n // 2$ on is a leaf: its sift-down finds no children and returns at once. Correct, but $n/2$ wasted calls.

Checkpoint 6. (a) MinHeap: for it, $a < b$ **is** `_beats(a, b)`, so nothing changes. (b) It sifts a max-heap down as if it were a min-heap: the **smaller** child moves up, and the heap is broken whenever a sift-down runs — in every pop and in heapify. (c) **6 fail, 22 pass:** the [MaxHeap] cases of the tests that pop or build from a list — `test_pop_returns_sorted_order`, `test_invariant_holds_after_every_pop`, `test_heapify_from_arbitrary_order`, `test_is_valid_rejects_a_broken_array` and `test_interleaved_push_and_pop` — plus `test_max_heap_differs_from_min_heap`, where `MaxHeap([5, 1, 9]).peek()` returns 1. The push test passes because pushing never calls `_sift_down`.

Checkpoint 7. (a) $(2, \emptyset, b)$; then $(2, \emptyset, b) \rightarrow (2, 1, a)$; then $(1, 2, c) \rightarrow (2, 1, a) \rightarrow (2, \emptyset, b)$ — c sifts up to the root, and b moves to index 2. (b) **c, b, a.** c has priority 1. b and a tie at 2; b was enqueued first (counter 0 against 1), so it comes out first. (c) With `(priority, item)`: **c, a, b** — the tie is broken by comparing the strings, so a beats b alphabetically, whatever the order of arrival. With two dictionaries of equal priority: `TypeError: '<' not supported between instances of 'dict' and 'dict'`, on the second enqueue, when the sift-up compares them.

Checkpoint 8. $n = 1,024$: (a) about **2** (1.98), (b) about **8** (8.01), (c) about **2.3** (2.25). $n = 16,384$: 2.00, 12.00 and 2.27. Only (b) grows with n : descending values are the worst case for a min-heap, each new one climbing to the root, so the count per element is about $\log_2 n - 2$. Heapify stays at 2 or below — the summation of Lecture 12 — and random pushes stay flat because an average push stops near the bottom. heapify's advantage is its **worst case**.